SEION Math Core: A Reproducible Verification Suite for Finite-Dimensional N-Ary Algebra

Eliuth Chavero Jasso
Independent Researcher
Apizaco, Tlaxcala, Mexico
ORCID and email: to be supplied

Software and reproducibility companion, 29 July 2026

Abstract

SEION Math Core is a finite-dimensional verification suite for typed n-ary laws, ternary composition conventions, associator and symmetry diagnostics, projector reduction, finite kernels, truncated cohomology, tensor approximations, and reproducible numerical runs. This companion specifies the governance and evidence system that keeps mathematical statements, software behavior, and empirical observations separate. It defines a durable memory contract, authority ladder, action gate, run artifact contract, deduplicated scientific index, claim/evidence matrices, and fail-closed release procedure. The current audit preserves 73 historical run records and derives 9 scientific instances, with 8 duplicate groups and 64 repeated records; all historical records satisfy the declared artifact contract. Fast and full canonical profiles pass their software checks, including a CUDA execution path, but the release remains yellow because the theorem program, multi-seed bound-tightness experiments, and author contact metadata are incomplete. The repository therefore supplies a reproducible verification foundation, not a claim of universal mathematical validity.

Keywords: reproducible computational mathematics; evidence ledger; n-ary algebra; scientific software; governance; finite-dimensional verification.

1 Purpose and scope

The companion exists so that the software contribution is independently auditable from the theorem-focused manuscript. It owns repository architecture, schemas, commands, hardware manifests, run indexes, figure and table provenance, recovery procedures, and release bundles. It does not turn a passing numerical run into a proof and does not use repository terminology as a substitute for novelty. Its projection and tensor vocabulary is positioned against established numerical literature rather than presented as a new decomposition method [2, 1].

The system is intentionally local to the SEION Math Core repository. The external directories that inspired the architecture are not dependencies, edit targets, or sources of runtime state. The source-of-truth hierarchy is encoded in `AGENTS.md`, `governance/AUTHORITY_LADDER.yaml`, and `.ai/MEMORY_MANIFEST.yaml`.

2 Architecture

The repository is organized around five separations:

Table 1: Core contracts and their evidence role.

Contract	Canonical location	Failure interpretation
Source authority	`governance/AUTHORITY_LADDER.yaml`	Lower-authority notes cannot overwrite claims or theorem records.
Memory manifest	`.ai/MEMORY_MANIFEST.yaml`	Missing durable state blocks context recovery.
Run artifact	`src/seion_core/governance/runs.py`	A run without manifest, metrics, config, or logs is incomplete.
Evidence event	`schemas/evidence_event.schema.json`	Evidence must retain authority, commit, artifacts, and limitations.
Action policy	`governance/ACTION_POLICY.yaml`	High-risk or prohibited actions are denied or require human approval.
Release gate	`scripts/release_bundle.ps1`	Strict yellow/red audit stops packaging.

1. **durable memory**: human-readable decisions, tasks, blockers, risks, handoff notes, and recovery rules under `.ai/`;
2. **governance**: machine-readable authority, action, lifecycle, agent, and research/software contracts under `governance/`;
3. **scientific source**: claims, theorem registries, experiment configurations, and paper sources;
4. **derived evidence**: run manifests, generated figures, tables, indexes, quality reports, and the append-only event ledger;
5. **delivery**: paper PDFs and release bundles, produced only after the appropriate audit.

The executable boundary is `src/seion_core/governance/`. Its modules implement run collection and deduplication, evidence events, action gates, bounded context packs, post-flight handoffs, conservative language checks, and structural audits. The CLI exposes these through `seion-coregovernance...`.

3 Contracts and authority

The authority ladder distinguishes canonical source, verified derived artifact, registered observation, working interpretation, and unverified note. Every memory update requires provenance fields: source, observation time, command or artifact, and limitation. The action gate prohibits deleting evidence, rewriting history, pushing directly to the protected branch, publishing official claims, approving theorems, and labeling numerical evidence as proof.

4 Development lifecycle

The lifecycle is: recover context; register a task and its claims; make the smallest local change; run the narrowest relevant test; execute the fast or full canonical profile; rebuild derived artifacts; audit claims, runs, memory, paper, and language; append a post-flight handoff; and package only if the strict gate

Table 2: Current run-index audit.

Metric	Value
Historical records	73
Unique scientific instances	9
Duplicate groups	8
Repeated records excluded from unique view	64
Records with complete artifact contracts	73
Complete runs	71
Runtime failures retained for audit	2

is green. The state machine is recorded in `governance/DEVELOPMENT_LIFECYCLE.yaml`. Runtime state is kept out of durable memory through `.gitignore` rules for `.ai/runtime`, `.ai/machine`, and derived context packs.

5 Run identity and deduplication

Repeated execution of one matrix row is not counted as independent science. The derived identity is

$$(\text{experiment id}, \text{resolved-config hash}, \text{seed}, \text{precision}, \text{backend}, \text{device}).$$

The historical index is preserved for provenance; `artifacts/index/run_index_deduplicated.csv` is the analysis view. The current audit is summarized in Table 2.

These counts are a snapshot of the registered repository state. A deduplicated index prevents an inflated sample count but does not establish independence of the data-generating process.

6 Evidence and epistemic language

The append-only ledger `.ai/evidence/ledger.jsonl` records event identifiers, authority levels, commit and branch metadata, commands, artifact paths, outcomes, and limitations. The language audit searches for proof verbs combined with numerical or empirical markers and raises a warning when prose risks conflating the two. The paper generator emits claim status macros so that a table or caption can distinguish proved, tested, observed, conjectural, open, and refuted statements.

7 Verification commands

The following commands are the canonical entry points from the repository root:

```
python -m pytest -q
powershell -ExecutionPolicy Bypass -File scripts\run_fast.ps1
powershell -ExecutionPolicy Bypass -File scripts\run_full_blackwell.ps1
powershell -ExecutionPolicy Bypass -File scripts\build_all_artifacts.ps1
powershell -ExecutionPolicy Bypass -File scripts\build_paper.ps1
python -m seion_core.cli.main governance audit --strict --json
```

The first four commands are validation and artifact-generation steps. The strict audit is intentionally last: it is an approval predicate, not a replacement for tests. The current fast and full profiles pass their declared software checks; the full profile records a CUDA path on an NVIDIA RTX PRO 5000 Blackwell Generation Laptop GPU with peak GPU memory of approximately 35.9 MB in the observed run.

8 Figures, tables, and paper split

Generated scientific assets are rebuilt by `scripts/build_artifacts.py`. The centralized style module `scripts/figure_style.py` sets typography, print-safe colors, vector PDF output, and the absence of overlaid provenance text. Every quantitative figure must be traceable to a registered artifact and must state its metric, baseline, seeds, and limitation. The current suite still contains illustrative finite curves; the audit keeps this as blocker B-0003 rather than presenting them as asymptotic evidence.

The theorem-focused source is `papers/foundations/main.tex`. This companion is `papers/software/main.tex`. The older broad release note remains under `paper/main.tex` and is built by the existing paper pipeline; it is not silently relabeled as the stronger research paper.

9 Release policy and current status

The release script invokes `seion-core governance audit -strict -json` before creating a bundle. A yellow result is useful for development but fails the release command. The present audit is yellow, with no structural errors, because it reports duplicate historical executions and the paper quality flag is not release-ready. The four registered scientific blockers are:

1. B-0001: integrate and independently review the central theorem program;
2. B-0002: retain the historical index but use the deduplicated view for scientific summaries;
3. B-0003: replace illustrative curves with registered multi-seed experiments and bound-tightness ratios;
4. B-0004: supply verified ORCID and email metadata.

This fail-closed state is intentional. It makes the software system operational without misrepresenting the scientific maturity of the manuscript.

10 Reproducibility manifest

Source, governance, memory, schemas, and derived run indexes are included by `scripts/release_bundle.py`. Machine-local runtime state is excluded. The audit writes:

- `artifacts/index/governance_audit.json`;
- `artifacts/index/memory_health.json`;
- `docs/reviewer_report.md`.

The handoff and run history are appended under `.ai/`. The repository can therefore be recovered from the durable manifest and regenerated artifacts without consulting any of the inspiration repositories.

References

[1] Peter Benner, Serkan Gugercin, and Karen Willcox. A survey of projection-based model reduction methods for parametric dynamical systems. *SIAM Review*, 57(4):483–531, 2015.

[2] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.